

Personalised Video Summarisation --- Build Log

Akash Kolath Nair

Phases 0--13, August 2026

Executive summary

This document logs all work completed on the Personalised Video Summarisation project, following the Step-by-Step Build Runbook, from an empty repository to a working end-to-end system.

Status: Phases 0--11 and 13 complete. Phase 12 (write-up hygiene) partial; Phase 10.3 (QFVS) and 11.3 (user study) not run.

The method in one line

For each generated persona p , a vision-language teacher (Qwen3-VL-8B) rescores every shot of a video from that persona's point of view; those scores replace gtscore in the standard eccv16 HDF5 file; a query-conditioned DSNet is then trained on the resulting persona datasets.

Headline findings

1. The pipeline works end to end. Raw .mp4 plus a free-text preference produces a personalised summary video. On five unseen test videos, two contrasting queries produced different summaries every time (mean Jaccard 0.643).
2. The auxiliary ranking objective is worth about 3x on the primary metric. Stock DSNet training reaches Kendall tau 0.083--0.090 on TVSum; the same architecture with a frame-level mse+rank loss reaches 0.158--0.270.
3. Concat conditioning cannot personalise; FiLM can. The runbook specifies concatenation as the default. Concat's query term is constant across frames, so it translates the feature sequence without reordering it. Measured: concat changes the summary for 0.4 per cent of persona pairs, FiLM for 27.4 per cent.
4. The teacher personalises, but not semantically. Qwen gives different personas different scores (divergence 0.48, 92 per cent of pairs differ), yet that divergence does not track persona semantic distance ($r = +0.022$, $p = 0.41$). The supervision is differentiated but not semantically organised --- and a student cannot learn a relationship its labels do not contain. This is the central limitation of the current method.
5. Teacher self-consistency ($k=3$) is not worth its cost. It produced no measurable improvement in label quality (paired $p = 0.382$) at three times the inference cost.
6. F1 is nearly useless on these benchmarks. A random-score model scores 56.42 F1 on TVSum against published DSNet's 62.09. Rank correlation is the only metric with discriminative range.

Hardware reality versus the runbook

The runbook assumes the HEX cluster with A100/H100 GPUs. The actual machine is nitt: a single shared box, no scheduler, three RTX 3090s (24 GB, Ampere sm_86).

Runbook assumption	Reality	Consequence
SLURM batch jobs	No scheduler	No pre-emption, but GPUs shared live
module load cuda	No module system	CUDA 12.5 already installed
FP8 checkpoints	sm_86 has no FP8	32B teacher impossible; 8B bf16 only
80 GB per GPU	24 GB	--max-model-len reduced 32768 -> 8192
Storage on request	25 GB home quota	All heavy data on local scratch

Phase-by-phase log

Phase 0--1: Repository and environments

Home directory was at 24.2 of 25.0 GB, leaving 0.8 GB --- insufficient for the two environments. /mnt/fast0 (the box's usual scratch, and where .bashrc pointed PIP_CACHE_DIR and HF_HOME) had zero bytes free, and the target directory did not exist, so downloads would have failed silently.

Resolution: environments and caches moved to /var/tmp/akn57 (local disk, 219 GB free, not counted against quota, benchmarked as fast as NVMe for small-file IO). An old 15 GB project was archived and removed, freeing quota to 3.64 GB used.

Two virtual environments, deliberately separate because vLLM pins torch versions that conflict with the training stack:

Environment	Size	Key pins
pvs-train	5.5 GB	torch 2.6.0+cu124, open_clip, h5py, ortools
pvs-vllm	11 GB	vllm 0.19.1, torch 2.10.0+cu128

Version selection was evidence-based, not assumed. vLLM moved to the CUDA 13 runtime at version 0.20.0, which requires driver ≥ 580 ; this machine has 555. Resolved without downloading by using `pip install --dry-run --report` to inspect the dependency graph. 0.19.1 is the newest usable version.

Phase 2: DSNet replication

Variant	Dataset	Ours	Published	Delta
Anchor-based	TVSum	62.09 +- 2.01	62.05	+0.04
Anchor-based	SumMe	47.23 +- 2.95	50.19	-2.96
Anchor-free	TVSum	61.16 +- 2.71	61.86	-0.70
Anchor-free	SumMe	51.31 +- 3.73	51.18	+0.13

Three of four within 0.7 points. The SumMe anchor-based gap is within one standard deviation of the split-to-split spread (SumMe has 25 videos; each fold tests on ~5).

DSNet targets Python 3.6--3.8, torch 1.1, numpy 1.19, ortools 8.0. Two library APIs had been removed in the interval. Total change: 14 insertions, 14 deletions across 3 files --- `np.bool` -> `bool` (numpy 2.0 removal) and the ortools knapsack module move. Both are pure API renames; no algorithm, hyperparameter or data path was altered.

Phase 3: Baselines

Method	TVSum	SumMe
Random floor	56.42 +- 0.90	41.49 +- 1.33
DSNet anchor-based	62.09	47.23
DSNet anchor-free	61.16	51.31
PGL-SUM	62.23	48.20

The random floor is the most important number here. A model assigning pure noise reaches 56.42 F1 on TVSum; every published architecture beats it by only 5--6 points, and all three cluster within 1.1 points of each other. F1 is computed after KTS segmentation and a knapsack filling a 15 per cent budget, so it largely measures the pipeline.

PGL-SUM was run on DSNet's splits (its own shipped splits overlap DSNet's by only 2--4 of 10 test videos) and scored with

DSNet's harness, after verifying line by line that the two codebases' summary-generation routines are algorithmically equivalent.

CLIP-It was dropped. The official repository has contained only a README saying "Code coming soon!" since 2021; the sole third-party implementation is 142 lines with no optimizer, loss, dataloader or evaluation. It would also have been blocked on CLIP features and captions.

Phase 4: CLIP feature re-extraction

75 videos re-encoded from 1024-d GoogLeNet to 512-d CLIP (ViT-B-32/laion2b_s34b_b79k), keeping every other dataset in the h5 byte-identical.

Does the feature swap cost accuracy? No:

Variant	Dataset	CLIP-512	GoogLeNet-1024	Delta
anchor-based	TVSum	61.16	62.09	-0.93
anchor-based	SumMe	49.86	47.23	+2.63
anchor-free	TVSum	61.29	61.16	+0.13
anchor-free	SumMe	50.64	51.31	-0.67

Mean absolute change ~1.1 F1, inside the split-to-split standard deviation. This matters for the thesis: the move to CLIP is required for query conditioning and is not bought at the cost of feature quality.

The mapping problem. picks index the original video, and the TVSum h5 stores no filename. Pairing the wrong mp4 with the right gtscore is silent --- training runs, loss falls, every number is wrong. The previous attempt at this project left a file named segment_captions_MISMATCHED.json.bak.

Mapping was derived from evidence and verified before any encoding: SumMe by video_name plus frame-count confirmation, TVSum by exact frame count from ffprobe -count_frames. Result: TVSum 50/50, SumMe 25/25.

Verification used a rank test: the base GoogLeNet features came from the correct video at the same picks, so the CLIP features' frame-to-frame similarity profile should match. All 75/75 videos ranked #1 against their own base profile; a deliberately wrong pairing scores +0.005.

Phase 5: Serving the teacher

Qwen3-VL-8B-Instruct (bf16) on one RTX 3090.

Memory was computed from the model config before downloading: 36 layers x 8 KV heads x 128 head_dim = 144 KB per token. With 16 GB of weights on a 24 GB card, the runbook's --max-model-len 32768 would allow 0.87 concurrent sequences --- less than one request. Using 8192 gives 3.47x concurrency and fits the real workload (16 images = 3,003 tokens). Measured after launch: 16.64 GiB weights, 3.9 GiB KV cache, 3.47x concurrency --- matching the prediction.

Three runbook instructions were wrong on this stack:

1. --limit-mm-per-prompt image=16 is invalid syntax in 0.19.1; it takes JSON.
2. extra_body={"guided_json": schema} is silently ignored --- it returns unconstrained plain text with no error. response_format must be used instead.
3. The score schema needs minItems/maxItems; without them the model legally returned an empty array [], which would have produced a video with no scores.

Does the teacher personalise? Tested directly. The first attempt failed: a pastry-chef persona rating a tyre-changing tutorial returned an all-zero constant vector. Two causes: domain mismatch, and a prompt that did not demand discrimination. With a domain-matched persona pair and a discrimination prompt, spearman between two personas' scores was +0.046 --- effectively uncorrelated orderings of the same video.

Phase 6: Persona generation

128 personas across 16 domains (TVSum's 10 native categories plus 6 assigned SumMe domains), each with a biography, occupation and natural-language preference_query.

Sampling uses strides co-prime with each attribute axis length, so N personas spread deterministically across the space.

The audit found three real defects, none visible by reading the JSON:

1. A stride bug collapsed watching_context to a constant within each domain (stride 5 on a 5-element list gives $(i*5) \% 5 == 0$ for all i).
2. Severe gender skew: 84 per cent she/her versus 9 per cent he/him when the model chose freely. Now sampled explicitly as an audited attribute: 34/37/29 per cent.
3. Occupation repetition (graphic designer x20); now 79 per cent unique.

A check not in the runbook: CLIP's text encoder truncates at 77 tokens silently, which would make over-long queries lose information and could collapse two personas to the same embedding. Measured max 48 tokens --- all fit.

Phase 7: Teacher scoring and persona datasets

(video, persona) pairs attempted	600
pairs scored	551
chunk calls (k=3)	6,576
persona h5 files	118 (721 MB)
wall time	~75 min scoring, ~6 min frame extraction

The chunking problem, not addressed by the runbook. Videos have up to 130 shots but only 16 images fit per request. The runbook says "chunk and stitch", but the model scores each chunk relative to that chunk, and the Phase 5 discrimination prompt forces every chunk to span the full 0--1 range. Measured on one 32-shot video, the three chunks each spanned ~0.1--0.9 while their true levels differed nearly threefold (means 0.49 / 0.41 / 0.18). Naive concatenation corrupts the global ordering --- exactly what the primary metric measures.

Fix: overlapping windows with least-squares anchor alignment on the shared shots.

48 pairs (8 per cent) were rejected by an std > 0 guard. They cluster by content, not shot count: Scuba, Bearpark_climbing, Excavators river crossing are continuous single-scene home videos where every shot genuinely looks alike.

Does it personalise? Pairwise spearman between two personas' gtscore for the same video (1,857 pairs): median 0.64; 27.3 per cent below 0.3; 15.3 per cent negative; but 20.5 per cent above 0.90. Personalisation is real but not uniform.

Phase 8: Query conditioning

QueryConditioner (concat / FiLM) fused into the frame features at the top of DSNet's forward pass. Total change for this phase: 18 insertions, 6 deletions. Cumulative DSNet diff: 32 insertions, 20 deletions across 5 files.

A correction to the runbook. It asks to verify the original model is recovered "with zero-init proj bias". That cannot hold: for concat, $\text{proj}([f; 0]) = W_f f + b$, and zeroing only b leaves a random linear map of the features. Both modes were instead initialised to the exact identity (concat: $W = [I \mid 0]$, $b = 0$; FiLM: $\gamma \rightarrow 1$, $\beta \rightarrow 0$), making the checkpoint verifiable as bit-identical (max difference 0.000e+00 for both models and both modes), and ensuring training starts from exactly the Phase 2 baseline.

Gradients were separately confirmed to reach the query path, since an identity initialisation could otherwise be dead.

Phase 9: Training the student

Final configuration: `--qcond film --qcond-lr-mult 100 --loss mse+rank --lambda-score 10 --lambda-rank 10 --lr 5e-5`.

Accuracy (spearman against held-out persona labels, 5 splits, TVSum):

Condition	labels	query	mean rho
A (the method)	persona	FILM	+0.2237
C (control)	generic	FILM	+0.2199

Paired A - C = +0.0037, $p = 0.872$ --- no accuracy difference.

Persona sensitivity (correlation between outputs for two different personas; 1.0 means the query is ignored):

Model	mean	% pairs < 0.9
A persona + FiLM	+0.9080	27.4%
C generic + FiLM	+0.9829	1.0%

Persona supervision produces a persona-sensitive model --- 27 times more differentiated outputs than the control --- while producing no gain in aggregate accuracy. The control behaves exactly as theory predicts: trained with queries on labels that do not depend on them, it learned to ignore the query.

Three bugs found during this phase:

1. The control was measuring itself. `evaluate()` scored each model against its own training target, so the generic-label control reported rho +0.4650 and appeared to beat the method 2.4x. Uncorrected, Phase 9 would have concluded the opposite of the truth.
2. The query head never trained with concat: identity initialisation starts its weights at zero, and early stopping fired at epoch 0--2, leaving $|W_{\text{query}}| = 7\text{e-}04$.
3. Loss weighting. DSNet's detection losses (~2.5) drowned MSE (0.11) and rank (0.66); weighting them 10x lifted rho from 0.140 to 0.192.

Phase 10: Evaluation

Otani protocol, implemented per dataset: TVSum correlates against each of 20 annotators then averages; SumMe averages the 15--18 binary annotator selections first. The random floor landing at tau +0.0009 confirms the implementation.

TVSum:

Method	tau	rho	F1
Human ceiling (leave-one-out)	0.3139	0.3957	---
Random floor	0.0009	0.0013	56.42
DSNet AF (GoogLeNet)	0.0898	0.1177	61.16
DSNet AF (CLIP)	0.0826	0.1079	61.29
Ours: persona + FiLM	0.1581	0.2059	---
Control: generic + FiLM	0.2701	0.3488	---

Two readings. First, the auxiliary ranking objective roughly triples tau over stock DSNet training. Second, this metric structurally penalises personalisation: the reference is the generic human annotation, so a model that deliberately deviates per persona is scored down for the behaviour it was built to have.

Synthetic-label validation (10.4, "the crux"). Teacher persona labels versus human annotators: tau +0.0802, 79.9 per cent of

pairs positive, against a random floor of +0.0009 and a human ceiling of +0.3139. The labels are not noise --- 80 times the floor --- but capture only 26 per cent of human-ceiling agreement.

Phase 11: Ablations and persona sensitivity

Axis 1 --- query mechanism (the decisive axis):

Variant	rho	persona sens.	% pairs < 0.9
FILM	+0.2237	+0.9080	27.4%
concat	+0.2036	+0.9923	0.4%
none	+0.2025	---	---

Concat is functionally equivalent to having no query at all (+0.2036 versus +0.2025). This is structural: concat's query term is constant across frames, so it translates the sequence without reordering it, and rank metrics see only relative order.

Axis 2 --- loss:

Variant	rho	% pairs < 0.9
mse+rank	+0.2237	27.4%
rank only	+0.2209	16.6%
mse only	+0.2117	32.2%
none	+0.1648	10.6%

Axis 3 --- teacher self-consistency: k=1 tau +0.0823 versus k=3 +0.0802; paired p = 0.382. No benefit at three times the cost.

Persona sensitivity (11.2), the counterfactual test:

Condition	mean divergence	corr(distance, divergence)	p
TEACHER labels	0.4811	+0.0222	0.410
Ours: persona + FILM	0.1451	+0.0898	0.001
Control: generic + FILM	0.1082	+0.1564	0.000
Ours + shuffled queries	0.1787	+0.0333	0.218

The central finding. The teacher's labels diverge enormously (0.48, 92 per cent of pairs) but the divergence does not track persona semantic distance ($p = 0.41$). Semantically similar personas do not receive similar scores. The supervision is differentiated but not semantically organised, and a student cannot learn a relationship its labels do not contain.

The shuffled-query condition provides the positive signal: the distance-divergence correlation is significant only when the model receives the matching persona ($p = 0.001$) and vanishes when shuffled ($p = 0.218$). The model does respond to persona meaning, weakly but genuinely.

Phase 13: Raw video to personalised summary

Not in the runbook. `src/infer_personalised.py` takes an arbitrary .mp4 and a free-text preference and writes a summary video. The query is not restricted to the 128 generated personas --- CLIP maps arbitrary text into the same space.

Demo on TVSum video_21 (parkour), in split 0's test set, unseen, 647 s:

Query	Shots	Duration
"the athlete mid-jump and landing..."	45/175	97.0 s
"people being interviewed and talking to camera..."	44/175	97.0 s

Jaccard overlap 0.679. Across five unseen test videos, 5/5 produced different summaries, mean Jaccard 0.643.

Three bugs found: KTS degenerating on un-normalised CLIP features (635 "shots" for 636 frames); KTS vmax needing retuning from DSNet's 1.0 to 0.5 for CLIP's similarity distribution; and non-contiguous frames inheriting source timestamps, producing summaries that reported 376 s for 59 s of content.

Consolidated deviations from the runbook

#	Runbook says	Reality	Resolution
1	module load cuda	no module system	CUDA 12.5 preinstalled
2	FP8 32B teacher	sm_86 has no FP8	8B bf16 only
3	--max-model-len 32768	0.87 concurrent seqs	8192
4	--limit-mm-per-prompt image=16	invalid syntax	JSON form
5	guided_json	silently ignored	response_format
6	score schema without minItems	returns []	minItems/maxItems
7	"chunk and stitch"	corrupts global ordering	overlap + anchor alignment
8	zero-init proj bias recovers original	mathematically false	identity init
9	concat conditioning default	cannot personalise	FiLM
10	MSE against gtscore	DSNet is a detection model	auxiliary loss on pred_cls
11	self-consistency k=3 first	no measurable benefit	k=1 sufficient
12	clone CLIP-It	no implementation exists	dropped

Outstanding work

Item	Status	Note
Phase 10.3 QFVS	not run	UTE data never downloaded; the only protocol with genuine per-query ground truth
Phase 11.3 user study	not run	needs 10--15 human participants
Phase 9 on SumMe	not run	Phase 9 was TVSum-only; SumMe rows are zero-shot transfer
Phase 12 versioning	partial	1 commit; 40 files uncommitted
PGL-SUM rank rows	not computed	per-frame scores exist in saved epoch JSONs

Highest-value next experiment: QFVS. Phase 10 showed that on TVSum and SumMe a personalised model can only be penalised for personalising, because their ground truth is generic. QFVS is the only dataset that can adjudicate the central claim.

Artefact inventory

Artefact	Size
Persona datasets (data/persona_h5/)	118 files, 721 MB
Teacher label cache (labels_cache/)	6,576 calls, 3.6 MB
Per-pair score files	551
Cached shot frames	3,117
Persona pool	128 personas, 16 domains
CLIP feature files	2 (tvsum, summe)
Trained checkpoints	90 (711 MB)
Project source	16 files, 2,846 lines
DSNet modifications	32 insertions, 20 deletions, 5 files
Home quota used	8.62 of 25.0 GB
Local scratch used	56 GB

Reproduction

```
source env/activate-train.sh          # DSNet / CLIP / evaluation
source env/activate-vllm.sh          # vLLM teacher (needed only for Phase 6-7)

python src/map_videos.py --tvsum-dir ... --summe-dir ...
python src/extract_clip.py --dataset tvsum
python src/verify_clip_h5.py --dataset tvsum summe
python src/build_seeds.py
python src/gen_personas.py --n-per-domain 8
python src/audit_personas.py
python src/extract_shot_frames.py
python src/score_teacher.py --k 3 --workers 10
python src/build_persona_h5.py
python src/query_head.py --precompute
python src/train_persona.py --qcond film --qcond-lr-mult 100 --loss mse+rank \
    --lambda-score 10 --lambda-rank 10
python src/eval_rank.py --dataset tvsum
python src/persona_sensitivity.py
python src/infer_personalised.py --source video.mp4 --query "..." --save out.mp4
```

Every teacher call is cached, so re-running Phase 7 costs nothing.

Appendix: full phase notes

The remainder of this document reproduces the per-phase notes written at the time, unedited. They contain the full measurements, command lines and reasoning.

Phase 1 -- Environment notes

Date set up: 2026-08-11. Machine: nitt.

This file records how the actual environment differs from the Build Runbook, which was written for the HEX cluster. Every deviation below is forced by the hardware or storage on this machine, not a preference.

1. The machine is not HEX

Runbook assumes	Reality on nitt
HEX cluster, SLURM batch jobs	Single shared box, no scheduler (sbatch/squeue absent)
module load cuda/12.4	No module system. CUDA 12.5 already at /usr/local/cuda-12.5
A100/H100, 80 GB, FP8 capable	3 x RTX 3090, 24 GB each, Ampere sm_86
Storage quota on request	Home quota 25 GB, hard-capped

Consequences:

- No pre-emption (there is no queue), so the runbook's "checkpoint the teacher job against pre-emption" is less critical -- but labels_cache/ is still worth it, because the box is shared and someone else's job can OOM the GPU you are using.
- You share GPUs live with other users. Always set CUDA_VISIBLE_DEVICES and never take all three cards.

2. FP8 is impossible on this hardware

vLLM reports directly:

```
compute cap : DeviceCapability(major=8, minor=6)
supports fp8 : False
```

FP8 requires Hopper/Ada (sm_89+). Ampere sm_86 has no FP8 tensor cores.

Phase 5 impact: Qwen/Qwen3-VL-32B-Instruct-FP8 cannot run here at all.

- Workhorse teacher: Qwen3-VL-8B-Instruct in bf16 (~16 GB) fits one 3090 (24 GB).
- The 32B ablation (Phase 11.1) needs a different route: a 4-bit AWQ/GPTQ 32B across 2 GPUs with --tensor-parallel-size 2. Not yet tested.

3. Storage layout

Home is quota-capped at 25 GB -- far too small for two envs (~17 GB) plus model weights (~16 GB per model). /mnt/fast0 (the box's usual scratch, and where .bashrc used to point) has 0 bytes free; /mnt/faster0 has only ~42 GB at 98% full and is shared. So everything heavy lives on the local disk:

```
/var/tmp/akn57/          # local disk (sda2), ~190 GB free, NOT on your quota
??? envs/pvs-train/      # 5.5 GB  training env
??? envs/pvs-vllm/       # 11 GB   inference env
??? cache/{pip,huggingface,torch}/ # was pointed at the full /mnt/fast0
??? old-project/         # 7.7 GB  copy of the deleted ~/Personalised-Video-Summary
??? backup/
    ??? pvs-old-project_BACKUP.tar.gz      # 237 MB, irreplaceable work only
    ??? pvs-old-project_BACKUP.tar.gz.sha256
    ??? MANIFEST.txt
    ??? patches/                          # DSNet + PGL-SUM local modifications as .patch
```

Caveat: /var/tmp is local to nitt. It does not follow you to another machine, and it is not backed up. /var/tmp has no auto-delete rule on this box (only /tmp does, 30 days) -- but the envs are rebuildable from the lockfiles here, so a loss costs

one rebuild, not any work.

.bashrc was updated (backup in backup/) to point HF_HOME, TORCH_HOME and PIP_CACHE_DIR at this scratch instead of the full /mnt/fast0.

4. Version choices and why

Training env (pvs-train) -- torch 2.6.0+cu124

Driver 555.42.06 supports CUDA 12.5. PyTorch channels offered:

Channel	Newest for py3.12	Verdict
cu124	2.6.0 stable	chosen -- 12.4 <= 12.5, guaranteed
cu126	only 2.13.0.dev nightlies	rejected: never pin a dissertation to a nightly
cu128	2.11.0 stable	rejected here: needs forward-compat, and 2.6 is ample for DSNet

Inference env (pvs-vllm) -- vllm 0.19.1, torch 2.10.0+cu128

vLLM moved to the CUDA 13 runtime at version 0.20.0. CUDA 13 requires driver >= 580; this box has 555. Resolved without downloading via pip install --dry-run --report:

vLLM	torch	cuda-runtime	usable here
0.11.0 - 0.19.1	2.8.0 - 2.10.0	12.8.90 (cu12)	yes
0.20.0+	2.11.0+	13.0.96 (cu13)	no -- driver too old

0.19.1 is therefore the newest usable vLLM, and is well above the runbook's 0.11.0 minimum for Qwen3-VL. Do not upgrade past 0.19.1 unless the driver is upgraded first.

cu128 on a cu125 driver relies on CUDA minor-version compatibility; verified empirically (bf16 matmul on GPU succeeds), not assumed.

5. Reproducing these environments

```
## training env
python3 -m venv /var/tmp/akn57/envs/pvs-train
source /var/tmp/akn57/envs/pvs-train/bin/activate
pip install torch==2.6.0 torchvision==0.21.0 --index-url https://download.pytorch.org/whl/cu124
pip install -c env/constraints-train.txt h5py scipy scikit-learn tqdm pyyaml ortools open_clip_torch ftfy
regex

## inference env
python3 -m venv /var/tmp/akn57/envs/pvs-vllm
source /var/tmp/akn57/envs/pvs-vllm/bin/activate
pip install vllm==0.19.1 qwen-vl-utils==0.0.14 openai pillow
```

Exact pins: requirements-train.txt (58 pkgs), requirements-vllm.txt (181 pkgs). The constraints-*.txt files pin torch so a later pip install cannot silently swap in a PyPI torch built for the wrong CUDA -- a failure mode that is very hard to diagnose.

6. Checkpoint results (Phase 1, passed)

```
train env : torch 2.6.0+cu124, cuda available True, 3x RTX 3090 sm_86,
           GPU matmul OK, ortools knapsack solves, all 12 imports OK
vllm env  : vllm 0.19.1 (>= 0.11.0 required), torch 2.10.0+cu128,
           cuda available True, bf16 matmul OK, supports_fp8 False
```

Phase 2 -- DSNet reproduction (baseline #1)

Date: 2026-08-11. Machine: nitt, RTX 3090 (GPU 0 = anchor-based, GPU 1 = anchor-free). DSNet base commit: 1804176e2e8b57846beb063667448982273fca89.

Result

F1 (%), mean +- std over the repo's own 5 splits, evaluated with evaluate.py:

Variant	Dataset	Ours	Published	Delta
Anchor-based	TVSum	62.09 +- 2.01	62.05	+0.04
Anchor-based	SumMe	47.23 +- 2.95	50.19	-2.96
Anchor-free	TVSum	61.16 +- 2.71	61.86	-0.70
Anchor-free	SumMe	51.31 +- 3.73	51.18	+0.13

Three of four land within 0.7 points. The one gap is anchor-based on SumMe (-2.96), and it is within one standard deviation of the split-to-split spread (+2.95). SumMe has only 25 videos, so each of the 5 folds tests on ~5 videos; per-split F1 ranges from 43.24 to 51.12 for this cell. A 3-point difference at that sample size is noise, not a defect.

Per runbook 2.3, these local numbers are the reference baseline for all later comparisons, not the published ones. Full per-split values are in ../baselines.csv; raw logs in train_.log and eval_.log.

evaluate.py reproduced the training-time F-scores exactly, which confirms checkpoints round-trip correctly through save/load.

Changes made to DSNet

The runbook says "clone, unmodified". DSNet targets Python 3.6-3.8, torch 1.1.0, numpy 1.19.5, ortools 8.0; this machine runs Python 3.12, torch 2.6, numpy 2.4, ortools 9.15. Two library APIs had been removed in the interval.

Complete diff: 14 insertions, 14 deletions across 3 files -- saved as dsnet_compat_fixes.patch, applies cleanly to base commit 1804176.

Fix	Files	Cause
np.bool -> bool (8 sites)	vsumm_helper.py, bbox_helper.py, anchor_free_helper.py	numpy 2.0 removed the deprecated alias
ortools.algorithms.pywrapknapsack_solver -> ortools.algorithms.python.knapsack_solver; Init/Solve/BestSolutionContains -> init/solve/best_solution_contains	vsumm_helper.py	ortools 9.x moved the module and renamed methods to snake_case

Every change is a library API rename. No algorithm, hyperparameter, loss, split, or data path was altered. For the dissertation this is defensible as "DSNet unmodified apart from two library-compatibility renames forced by a five-year dependency gap."

Verified before training, against known-correct values:

```
knapsack([60,100,120],[10,20,30], cap=50) -> items [1,2], value 220 (optimal)
f1_score([1,1,0,0],[1,0,0,0])             -> 0.6667 (correct)
get_keyshot_summ(...)                      -> selects 15.0% of frames (standard budget)
```

torch.load was deliberately not changed. torch 2.6 flipped weights_only to True by default, but these files are plain tensor state_dicts, which load fine -- confirmed by evaluate.py reproducing the training F-scores from disk.

Notes for later phases

--num-feature 1024 matches GoogLeNet pool5. Phase 4 replaces features with 512-d CLIP, so training must pass --num-feature 512 or it fails on a shape mismatch.

- torch_geometric is never needed with the default --base-model attention; the import lives inside GCNExtractor.__init__. Do not use --base-model gcn -- it would drag in torch-scatter/sparse/cluster, which usually need compiling from source.

- Training cost: ~2h for 10 splits (5 folds x 2 datasets) at 300 epochs, per variant.

The two variants ran concurrently on separate GPUs. GPU memory ~1 GB, utilisation <20% -- the bottleneck is data loading, not compute, so parallel runs are near-free.

- Seed is fixed at 12345 in the repo defaults; runs are reproducible.

Reproduce

```
source env/activate-train.sh
cd third_party/DSNet/src
export CUDA_VISIBLE_DEVICES=0
python train.py          anchor-based  --model-dir  ../../../../models/ab_basic  --splits  ../splits/tvsum.yml
../splits/summe.yml
python evaluate.py       anchor-based  --model-dir  ../../../../models/ab_basic  --splits  ../splits/tvsum.yml
../splits/summe.yml
python train.py          anchor-free   --model-dir  ../../../../models/af_basic  --splits  ../splits/tvsum.yml
../splits/summe.yml --nms-thresh 0.4
python evaluate.py       anchor-free   --model-dir  ../../../../models/af_basic  --splits  ../splits/tvsum.yml
../splits/summe.yml --nms-thresh 0.4
```

Datasets downloaded fresh from the authors' Dropbox link (dsnet_datasets.zip, 178 MB):

```
a108d05ba7a9369ef6c891df475e6a2b  eccv16_dataset_summe_google_pool5.h5
b56ae0ae124d5b3f8e7c0f641bb5c059  eccv16_dataset_tvsum_google_pool5.h5
```

Phase 3 -- Baselines

Date: 2026-08-11. All numbers produced by DSNet's evaluation harness on DSNet's splits.

Results

F1 (%), mean +- std over 5 splits. "vs floor" is the gain over a model that assigns random importance scores.

TVSum

Method	F1	vs floor	Published	Delta
Random floor	56.42 +- 0.90	(floor)	-	-
DSNet (anchor-based)	62.09 +- 2.01	+5.67	62.05	+0.04
DSNet (anchor-free)	61.16 +- 2.71	+4.74	61.86	-0.70
PGL-SUM	62.23 +- 2.19	+5.81	61.0	+1.23

SumMe

Method	F1	vs floor	Published	Delta
Random floor	41.49 +- 1.33	(floor)	-	-
DSNet (anchor-based)	47.23 +- 2.95	+5.74	50.19	-2.96
DSNet (anchor-free)	51.31 +- 3.73	+9.82	51.18	+0.13
PGL-SUM	48.20 +- 6.80	+6.71	57.1	-8.90

The headline finding: the random floor

A model assigning pure noise scores reaches 56.42 F1 on TVSum. Every published architecture here beats it by only ~5-6 points, and all three cluster within 1.1 points of each other (61.16 - 62.23).

This is the Otani et al. (2019) result, reproduced locally on our own data and harness. The cause is structural: F1 is computed after KTS shot segmentation and a knapsack that fills a ~15% length budget. That pipeline produces a plausible summary regardless of the scores fed into it, so F1 largely measures the pipeline rather than the model.

Consequence for this project: reporting "the persona model achieves X F1" is close to meaningless without the floor beside it. This is the empirical justification for the runbook's decision to make rank correlation primary and F1 secondary ("never lead with it"). We now have that evidence first-hand rather than by citation.

Random floor is averaged over 5 seeds (a single random draw is itself noisy): TVSum per-seed 56.09 / 57.48 / 56.06 / 57.18 / 55.28.

Methodological decisions

Two choices that a reviewer will probe, both made for comparability:

1. PGL-SUM was run on DSNet's splits, not its own. Both repos ship "5 randomly generated 80/20 splits", but they were generated independently: their test sets overlap by only 2-4 of 10 videos per fold. Comparing across them would compare test sets, not models. DSNet's splits were converted to PGL-SUM's JSON format (third_party/PGL-SUM/data/datasets/splits/).

2. PGL-SUM was scored with DSNet's harness, and with DSNet's epoch-selection rule. DSNet keeps the epoch with max F-score on the test split (anchor_based/train.py:102-104). That is optimistic (test-set model selection), but applying a stricter rule to PGL-SUM alone would rig the comparison. Both therefore use max-over-epochs. Epoch -1 (untrained) excluded. Implemented in src/eval_pglsun.py.

Before trusting this, the two codebases' summary-generation routines were compared line by line and are algorithmically equivalent:

Step	PGL-SUM <code>generate_summary.py</code>	DSNet <code>get_keyshot_summ</code>
upsample subsampled scores to frame level	via positions	via picks
shot score	mean of frames in shot	mean of frames in shot
selection	knapsack, 15% budget	knapsack, 15% budget

Only difference: DSNet quantises shot scores to $\text{int}(1000 * \text{mean})$ because ortools requires integer values. That cannot account for a multi-point difference.

The SumMe discrepancy (PGL-SUM, -8.9)

TVSum reproduces well (+1.23). SumMe does not (48.20 vs 57.1 published). Contributing factors, in order of likely importance:

1. Different splits. The published number uses PGL-SUM's own splits; we used DSNet's (see decision 1). On a 25-video dataset, split choice moves the number a lot.
2. Extreme variance. SumMe has 25 videos, so each fold tests on ~5. Our per-split results range 40.91 to 57.84, std 6.80. The published 57.1 sits within ~1.3 std of our mean -- and is almost exactly our best split (57.84).
3. Split 2 failed to train: its best epoch was 0 of 200, i.e. the model never beat its initialisation on that fold.

Not yet ruled out: whether PGL-SUM recovers ~57 on its own splits. That is a cheap, worthwhile check (10 runs, ~45 min) and would cleanly separate "splits" from "our setup". Not run yet.

Per the Phase 2 convention, our locally produced numbers are the reference baseline.

CLIP-It: dropped

Runbook 3.2 assumes a clonable reference implementation. There isn't one:

- Official medhini/clip_it: README only, "Code coming soon!" since 2021.
- Unofficial srpkdyy/CLIP-It: 142 lines, one file, model definition only. No optimizer, loss, dataloader or eval (grep for optimizer|loss|backward|DataLoader|train(returns zero matches). Expects raw frames and 7 caption sentences, not the precomputed h5 features.

It would also have been blocked on CLIP features (Phase 4) and captions (Phase 5). Dropped by decision on 2026-08-11. The clone is left in `third_party/CLIP-It/` unused.

Replacement control. CLIP-It's role was to separate "the model reads a query" from "the supervision is persona-conditioned". A cleaner substitute, planned for Phase 9: train the Phase 8 query-conditioned DSNet on the stock gtscore. Same architecture, query head enabled, generic supervision -- an apples-to-apples ablation within one model instead of across two codebases. Costs one training run, no new code.

Files

<code>src/random_baseline.py</code>	random floor (5 seeds, DSNet harness)
<code>src/eval_pglsun.py</code>	PGL-SUM scores -> DSNet harness
<code>src/watch_progress.sh</code>	live training progress
<code>results/baselines.csv</code>	the table
<code>results/phase2/random_floor.json</code>	per-seed random results
<code>results/phase3/pglsun_dsnet_harness.json</code>	per-split PGL-SUM results + best epochs
<code>results/phase3/pglsun_*.log</code>	raw training logs (tqdm noise, no metrics)

Reproduce

```
## random floor
source env/activate-train.sh
cd third_party/DSNet/src
python ../../src/random_baseline.py --splits ../splits/tvsum.yml ../splits/summe.yml --seeds 5

## PGL-SUM (10 runs; TVSum ~35 min, SumMe ~15 min with 5 concurrent per GPU)
cd third_party/PGL-SUM
for i in 0 1 2 3 4; do CUDA_VISIBLE_DEVICES=1 python model/main.py --split_index $i --n_epochs 200
--video_type TVSum & done
for i in 0 1 2 3 4; do CUDA_VISIBLE_DEVICES=0 python model/main.py --split_index $i --n_epochs 200
--video_type SumMe & done
wait
cd ../../
python src/eval_pglsun.py --exp third_party/PGL-SUM/Summaries/PGL-SUM/exp1 --out
results/phase3/pglsun_dsnet_harness.json
```

PGL-SUM required no compatibility fixes (runs clean on torch 2.6). It needs only tensorboardX; the tensorflow import lives in evaluation/exportTensorFlowLog.py, which drives their epoch-selection and is unused here.

Phase 4 -- CLIP feature re-extraction

Date: 2026-08-12. Model: open_clip ViT-B-32 / laion2b_s34b_b79k, 512-d.

Why

Phase 8 conditions DSNet on a persona query encoded by CLIP's text encoder. CLIP is trained so image and text embeddings share one space, so a query and a frame become directly comparable. GoogLeNet pool5 has no text counterpart, so there would be nothing to compare the query against.

Only features changes (1024-d GoogLeNet -> 512-d CLIP). gtscore, change_points, picks, n_frames, n_steps, user_summary, gtsummary and n_frame_per_seg are copied through unchanged, so KTS, knapsack selection and the evaluation harness are untouched.

Does switching features cost accuracy?

Same splits, same harness as Phase 2 -- the only variable is the features.

Variant	Dataset	CLIP-512	GoogLeNet-1024	Delta
anchor-based	TVSum	61.16	62.09	-0.93
anchor-based	SumMe	49.86	47.23	+2.63
anchor-free	TVSum	61.29	61.16	+0.13
anchor-free	SumMe	50.64	51.31	-0.67

Conclusion: CLIP features are equivalent to GoogLeNet for this task -- mean absolute change ~1.1 F1, in both directions, well inside the split-to-split std (2-4 points). This matters for the thesis: the move to CLIP is required for query-conditioning and it is not bought at the cost of feature quality, so Phase 9 gains cannot be dismissed as an artefact of changing the backbone. Both remain ~5 points above the random floor (TVSum 56.42 / SumMe 41.49), the same margin as GoogLeNet.

Caveat -- ignore the diversity column when comparing across features. It jumps from ~0.47 (GoogLeNet) to ~75 (CLIP) simply because it is computed from raw feature vectors and CLIP embeddings have a much larger norm (L2 ~11). It is not comparable across feature spaces. F1 is unaffected.

The mapping problem (the real risk of this phase)

picks index the original video, and the h5 gives no filename for TVSum (video_name exists for SumMe only). Pairing the wrong mp4 with the right gtscore is silent: training runs, loss falls, every number is wrong. The previous attempt at this project left a file named segment_captions_MISMAPPED.json.bak.

Mapping is therefore derived from evidence and verified before any encoding (src/map_videos.py, kept deliberately separate from extraction):

- SumMe -- match video_name to filename, then confirm exact frame count.
- TVSum -- no names stored, so match on n_frames from ffmpeg -count_frames (decoded count, not the unreliable container header).
- Hard check for every video: max(picks) < decoded_frame_count.

Result: TVSum 50/50, SumMe 25/25.

Three problems it caught

1. SumMe ships every video twice (.mp4 and .webm, identical stems). Name matching alone picks one arbitrarily. Now the candidate whose decoded frame count equals the h5's is chosen -- video_25 legitimately resolves to playing_ball.webm.
2. Two TVSum videos had no exact match: video_16 -> WG0MBPpPC6I.mp4 and video_39 -> Se3oxnaPsz0.mp4, each +1 frame (2019-vs-2026 decoder difference). The alternative pairing was off by ~5,000

frames, so unambiguous. Handled by a documented ± 2 tolerance pass that still refuses genuinely ambiguous cases.

3. The verification itself was wrong (see below).

Verification: the rank test

src/verify_clip_h5.py checks schema preservation, shape/sanity, and mapping.

The mapping check exploits the fact that the base GoogLeNet features were computed from the correct video at the same picks. So the frame-to-frame cosine similarity profile of the CLIP features should track the GoogLeNet one.

First version used an absolute correlation threshold, and it was wrong. It flagged SumMe video_1 (Air_Force_One) at 0.246. Investigation: that video is nearly static -- its GoogLeNet profile has mean 0.975 and std 0.015, so consecutive frames are ~97.5% identical and there is almost no temporal variation to correlate. The coefficient is small even though the mapping is correct.

Replaced with a rank test, which is scale-free: does this CLIP profile match its own base profile better than every other video's? For video_1 the answer was yes -- rank #1 of 25 (0.246 vs next best 0.180) -- and video_name plus an exact frame-count match agreed independently.

Final result:

	TVSum	SumMe
videos	50	25
feature dim	512	512
non-feature datasets bit-identical	yes	yes
rank-#1 matches	50/50	25/25
mean profile corr	0.786	0.599
control (deliberately wrong pairing)	+0.005	+0.055

Decisions

- Sequential decode (PyAV), never frame seeking. OpenCV's CAP_PROP_POS_FRAMES can silently return a neighbouring frame on some codecs, misaligning every feature with its label while looking normal. Sequential decode is slower and exact.
- Copy-then-replace, so non-feature datasets are provably untouched.
- Resumable -- already-extracted videos are skipped.
- Each video group records clip_model and source_video attrs for provenance.

Files

```
data/clip_h5/tvsum_clip.h5    120 MB, 50 videos, features (n_steps, 512)
data/clip_h5/summe_clip.h5   36 MB, 25 videos
data/video_map.json          verified h5 key -> mp4 mapping (+ match type)
src/map_videos.py            builds & verifies the mapping (run FIRST)
src/extract_clip.py          extraction
src/verify_clip_h5.py        schema + sanity + rank test
third_party/DSNet/splits/{tvsum,summe}_clip.yml
models/{ab_clip,af_clip}/    trained on CLIP features
results/phase4/*.log
```

Raw videos live at /var/tmp/akn57/data/raw/ (not redistributed, not in git): TVSum tvsum50_ver_1_1.tgz (641 MB) from people.csail.mit.edu/yalesong/tvsum, SumMe SumMe.zip (2.4 GB) from data.vision.ee.ethz.ch/cvl/SumMe.

Reproduce

```
source env/activate-train.sh
python src/map_videos.py --tvsum-dir /var/tmp/akn57/data/raw/ydata-tvsum50-v1_1/video \
```

```
--summe-dir /var/tmp/akn57/data/raw/videos --out data/video_map.json
python src/extract_clip.py --dataset tvsum      # ~15 min
python src/extract_clip.py --dataset summe      # ~14 min
python src/verify_clip_h5.py --dataset tvsum summe
cd third_party/DSNet/src
python train.py anchor-based --model-dir ../../models/ab_clip --num-feature 512 \
  --splits ../splits/tvsum_clip.yml ../splits/summe_clip.yml
```

For later phases

- --num-feature 512 is required on every DSNet run using the CLIP h5.
- ydata-tvsum50-info.tsv (from the TVSum download, at
/var/tmp/akn57/data/raw/ydata-tvsum50-v1_1/data/) gives each video's category and title -- directly usable for the Phase 6 domain-conditioned persona seeds (gap G2), avoiding hand-labelling 50 videos.
- The diversity metric is not comparable across feature spaces (see caveat above).

Phase 5 -- Serving Qwen3-VL as the teacher

Date: 2026-08-12. Model Qwen/Qwen3-VL-8B-Instruct (bf16), vLLM 0.19.1, one RTX 3090.

Working launch command

```
source env/activate-vllm.sh          # sets HF_HOME + VLLM_WORKER_MULTIPROC_METHOD=spawn
export CUDA_VISIBLE_DEVICES=1
vllm serve Qwen/Qwen3-VL-8B-Instruct \
  --port 8000 \
  --max-model-len 8192 \
  --gpu-memory-utilization 0.90 \
  --limit-mm-per-prompt '{"image": {"count": 16, "width": 512, "height": 512}}' \
  --served-model-name qwen3vl-8b
```

Startup ~5 min (196 s of it is engine init + CUDA graph capture). Measured:

model weights	16.64 GiB
available KV cache	3.9 GiB = 28,400 tokens
max concurrency at 8192 ctx	3.47x
16 images @ 512px	3,003 prompt tokens (~185/image)

Deviations from the runbook (all necessary)

1. --max-model-len 8192, not 32768. Predicted from the config before downloading: 36 layers x 8 KV heads x 128 head_dim x 2 (K,V) x 2 bytes = 144 KB/token. With 16 GB of weights on a 24 GB card only ~4 GB remains for KV cache, so 32768 tokens = 4.83 GB would allow 0.87 concurrent sequences -- less than a single request, i.e. thrash or OOM. 8192 gives 3.47x concurrency and comfortably fits the real workload (16 images + prompt + JSON ~ 3.5k tokens).
2. --limit-mm-per-prompt takes JSON in 0.19.1, not the runbook's image=16. The configurable form also caps resolution, which is what keeps image tokens bounded: '{"image": {"count": 16, "width": 512, "height": 512}}'.
3. FP8 is impossible (Phase 1 finding): sm_86 has no FP8 tensor cores, so the runbook's Qwen3-VL-32B-Instruct-FP8 cannot run here at all. 8B bf16 is the workhorse.

Structured output: the runbook's method silently fails

Method	Result on vLLM 0.19.1
extra_body={"guided_json": schema} (runbook 7.1)	'0.5, 0.8, 0.3' -- plain text. Schema ignored, no error, no warning
extra_body={"structured_outputs": {"json": schema}}	valid JSON
response_format={"type": "json_schema", ...}	valid JSON

guided_json was removed in newer vLLM and is now silently ignored. In Phase 7 this would yield thousands of unparseable responses, failing intermittently depending on what the model happened to emit.

Use response_format -- it is the OpenAI standard, so it survives a move to a hosted API, which is the whole reason for using the OpenAI-compatible server.

Schema design (two traps found)

Trap 1 -- an unconstrained array can come back empty. With the runbook's schema (no minItems), the model returned [] for a persona it judged irrelevant. finish_reason was stop and the JSON was valid, so nothing errored -- the video would simply have had no scores. Fix: "minItems": N, "maxItems": N.

Trap 2 -- array-of-objects wastes the token budget. The model pads JSON with large runs of whitespace; [{"shot_id":0,"importance":0.9}, ...] for 16 shots blew past 1200 tokens and truncated mid-array. A flat array of numbers, with position implying shot id (safe because the length is now pinned), costs 81 completion tokens for the same information.

Working schema:

```
N = len(shots)
schema = {"type": "array", "minItems": N, "maxItems": N,
          "items": {"type": "number", "minimum": 0, "maximum": 1}}
```

The core question: does the teacher actually personalise?

This is the assumption the whole project rests on, so it was tested directly on real TVSum shots (video_1 = "How to change tires for off road vehicles", category VT).

First attempt failed -- and the failure was informative:

persona	scores
pastry chef	all 0.00
sports fan	all 0.10
political journalist	0.00 ... 0.80 ... 0.80 (differentiated)

Constant scores are the runbook's "teacher scores all ~0.5" failure mode. A constant gtscore carries zero learning signal and makes rank correlation undefined.

Two causes, both fixable:

1. Domain mismatch. A pastry chef genuinely has no interest in any shot of a tyre-changing tutorial, so all-zero is arguably correct and useless. This is exactly why Phase 6 must key persona seeds by video domain (gap G2) -- personas must be drawn from the video's domain, never globally.
2. The prompt did not demand discrimination. Adding a system message requiring relative ranking ("rank shots RELATIVE to each other; at least one above 0.7 and one below 0.3; never return the same score for every shot") fixed the spread.

With a domain-matched persona pair and the discrimination prompt:

shot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
mechanic (wants exact steps)	.80	.70	.60	.30	.40	.20	.10	.50	.40	.30	.20	.10	.30	.90	.80	.70
impatient (wants the result)	.10	.10	.20	.10	.20	.10	.30	.10	.10	.10	.10	.10	.40	.70	.10	.10

- spearman(mechanic, impatient) = +0.046 -- effectively uncorrelated
- std 0.255 / 0.159 -- real spread, no degeneracy
- vs the human generic gtscore: mechanic +0.109, impatient -0.512 -- the persona scores deviate from the generic label, which is the point of the method

Conclusion: the teacher does personalise, conditional on (a) domain-matched personas and (b) a prompt that demands relative discrimination. Both are now requirements on Phase 6 and Phase 7, not optional quality levers.

Bonus for Phase 6

/var/tmp/akn57/data/raw/ydata-tvsum50-v1_1/data/ydata-tvsum50-info.tsv (from the Phase 4 download) gives every TVSum video's category, title and URL. TVSum is exactly 10 categories x 5 videos, which is the domain-seed structure Phase 6 needs -- no hand-labelling of 50 videos required.

Environment note

h5py and scipy were added to the vLLM env (constrained, torch/vllm unchanged): the scoring script must read change_points

from the h5 and call the server.

Checkpoint (runbook Phase 5)

- curl `http://localhost:8000/v1/models` -> lists `qwen3vl-8b`, `max_model_len 8192`.
- A chat request with a real TVSum frame returns an accurate description ("A man with gray hair ... against a background of horizontal blinds").
- 16-image structured-output request returns a valid 16-number array.

The server is still running on GPU 1 (PID 3423990) for Phases 6-7. Log: `results/phase5/vllm_server.log`.

Phase 6 -- Domain-conditioned persona pool

Date: 2026-08-12. Generator: Qwen3-VL-8B-Instruct via the Phase 5 server, temperature 0.9.

Output

```

personas/seeds.json  16 domains + attribute axes + video->domain map for all 75 videos
personas/pool.json   128 personas (8 per domain), each with persona_id, biography,
                    occupation, preference_query, attributes, domain, dataset

```

Domains (gap G2)

A single global persona distribution is wrong, and Phase 5 proved it: a pastry-chef persona rating a tyre-changing tutorial returned a constant all-zero score vector -- zero learning signal, undefined rank correlation. So personas are generated per domain and Phase 7 pairs a persona only with videos of its own domain.

- TVSum -- its own 10 native categories, 5 videos each, read from ydata-tvsum50-info.tsv and joined to h5 keys via the mp4 stem (== TVSum video_id): VT tyre repair, VU vehicles stuck, BK beekeeping, BT bike/motorcycle tricks, DS dog shows, GA animal grooming, MS making sandwiches, PK parkour, PR parades, FM flash mobs.

- SumMe -- no category field, so its 25 videos were assigned explicitly from their official titles into 6 domains: action_sports (10), aviation (4), vehicles (4), landmarks_travel (3), everyday_life (3), animals_nature (1). Hand-assigned but version-controlled in src/build_seeds.py and auditable.

animals_nature has only one video (Saving dolphins) -- too thin to support a per-domain claim on its own; treat it as merged or excluded in any SumMe-domain analysis.

Sampling: stride, not random

Random sampling of attribute combinations clumps and leaves holes. Each attribute axis is instead walked with a stride co-prime with the axis length, offset per domain, which spreads N personas evenly and is fully deterministic (so the pool regenerates identically).

Axes: age_band (5), attention_budget (4), expertise (4), watching_context (5), gender_presentation (3), plus per-domain viewing_goal and interest_focus. 1,200 global combinations before the domain-specific axes.

Temperature contrast worth remembering:

	temperature	why
persona generation (Phase 6)	0.9	diversity is the goal
shot scoring (Phase 7)	0.2	reproducible labels; variance is noise

The audit found three real defects

A pool can look perfect persona-by-persona and still be useless, so diversity was measured, not assumed (src/audit_personas.py).

1. A stride bug collapsed an axis. watching_context used only 3 of 5 values, and was constant within any single domain. Cause: stride 5 on a 5-element list gives $(i*5) \% 5 == 0$ for all i . attention_budget (4 values, stride 2) had the same defect. Fixed by selecting a stride co-prime with each axis length. All axes now reach full coverage in 8 draws.

2. Severe gender skew: 84% she/her vs 9% he/him when the model was free to choose. Fixed by sampling gender_presentation explicitly as an audited attribute -- now 34% / 37% / 29% (he / she / they). This controls the distribution rather than leaving it to the model's defaults, and does not affect what a persona wants to see.

3. Occupation repetition -- "graphic designer" x20, only 56% unique. Now 79% unique (101 distinct across 128), largest repeat "barista" x9.

Final pool quality

check	result
CLIP text length (limit 77, truncates silently)	mean 31.9, max 48 -- all 128 fit
pairwise cosine similarity (CLIP space)	mean 0.352, p95 0.580, max 0.849
near-duplicates (≥ 0.95)	0
worst within-domain mean similarity	0.618 (tvsum:VT)
best within-domain	0.376 (summe:everyday_life)
age bands	5 distinct, 22-32 each
expertise / attention budget	perfectly balanced, 32 each
occupations	101 distinct / 128 (79%)

The CLIP-token check is not in the runbook but matters: Phase 8 encodes preference_query with CLIP's text encoder, which truncates at 77 tokens without error. Over-long queries would silently lose information and could collapse two personas to the same embedding. Max is 48, so there is comfortable headroom -- but this must be re-checked if the query wording is ever expanded.

Similarity is measured in CLIP embedding space, not by string overlap, because that is the space Phase 8 actually conditions on.

Reproduce

```
python src/build_seeds.py # seeds.json (deterministic)
source /var/tmp/akn57/envs/pvs-vllm/bin/activate # needs the Phase 5 server up
python src/gen_personas.py --n-per-domain 8 --out personas/pool.json
source env/activate-train.sh
python src/audit_personas.py --pool personas/pool.json # needs open_clip => train env
```

Note the two-environment split: generation talks to the vLLM server, the audit needs open_clip, so they run in different venvs.

For Phase 7

- Pair a persona only with videos where seeds.json:video_domain[dataset][key] equals the persona's domain.
- Keep the Phase 5 discrimination system prompt and assert scores.std() > 0 before writing any persona h5 -- a constant vector is a silent data-quality failure.
- Workload if every persona scores every video in its domain:

TVSum 10 domains x 8 personas x 5 videos = 400 (video, persona) pairs; SumMe 6 domains x 8 personas x (its video counts) = 200 pairs. Videos with more than 16 shots need chunking, so the real call count is several times that -- hence the labels_cache/ requirement.

Phase 7 -- Teacher scoring and persona h5 files

Date: 2026-08-12. Teacher Qwen3-VL-8B-Instruct (bf16) via the Phase 5 server.

This is the core of the method: (video, persona) -> a gtsscore vector.

Output

```
labels_cache/*.json          6,576 cached chunk calls (the sacred cache)
labels_cache/scores/<ds>/<pid>/<video>.json  551 per-pair shot-score vectors
data/persona_h5/<ds>/<domain>/<persona_id>.h5 118 persona datasets, 721 MB
/var/tmp/akn57/data/shot_frames/ 3,117 cached shot frames (84 MB)
```

547 (video, persona) gtsscore vectors across 118 persona files, built on the CLIP h5 (Phase 8 conditions on CLIP text embeddings, so the features must be CLIP).

Scale and cost

(video, persona) pairs attempted	600
pairs scored	551 (48 rejected, see below)
chunk calls (k=3 self-consistency)	6,576
teacher throughput	3.47 calls/s at concurrency 6; ~10 workers to saturate
wall time	~75 min scoring + 6 min frame extraction

Frames are decoded once per video, not per pair. The persona changes the prompt, not the pixels; decoding inside the scoring loop would have meant ~600 decodes (~3 h) instead of 75 (~6 min).

The chunking problem (not addressed by the runbook)

The server allows 16 images per request; videos have up to 130 shots (TVSum median 38, max 130). The runbook says "chunk the shots and stitch". Naive stitching is wrong:

the model scores each chunk relative to that chunk, and the Phase 5 discrimination prompt -- which is required to avoid constant vectors -- forces every chunk to span the full 0-1 range.

Measured on a real 32-shot video, the three chunks each spanned ~0.1-0.9 while their true levels differed nearly 3x:

chunk	local range	mean
shots 0-15	0.10-0.92	0.49
shots 12-27	0.10-0.83	0.41
shots 24-31	0.07-0.90	0.18

Concatenating would rank the last chunk's local 0.90 alongside the first chunk's 0.92 -- corrupting the global ordering, which is exactly what the primary metric (rank correlation) measures.

Fix: overlapping chunks with anchor alignment. Windows of 16 shots with 4-shot overlap; each chunk is mapped onto the previous chunk's scale by a least-squares linear fit $a \cdot x + b$ over the shots they share, with a mean-matching fallback when the fit is degenerate. Overlapping shots average the two estimates. Implemented in `align_and_merge()` in `src/score_teacher.py`.

Note this makes absolute values comparable within a video. DSNet normalises gtsscore per video at load time (`data_helper.py:33-34`), so only the ordering matters downstream -- which is precisely what alignment protects.

48 rejected pairs (8%) -- the guard working

assert std > 0 rejects any constant score vector: it carries zero learning signal and makes rank correlation undefined. 48 pairs were dropped, and they cluster by content, not by shot count:

video	shots	degenerate personas
Scuba	15	8
Bearpark_climbing	23	7
Excavators river crossing	65	7
Saving dolphins	45	6
Fire Domino	11	6

Not a shot-count effect (video_12 has 7 shots and zero rejects). These are continuous single-scene home videos where every shot genuinely looks alike, so the teacher cannot discriminate. SumMe is unedited home footage; TVSum is edited YouTube content with real scene changes -- which is why rejects concentrate in SumMe.

Consequence: some personas cover fewer videos than their domain contains. Two summe:animals_nature personas were dropped entirely (that domain has only one video, already flagged in Phase 6).

Does it actually personalise? (honest answer)

Pairwise Spearman between two personas' gtscore for the same video, 1,857 pairs:

median	0.64
p10 / p90	-0.12 / 0.95
pairs with $r < 0.3$ (clearly different)	27.3%
pairs with $r < 0$ (opposed)	15.3%
pairs with $r \geq 0.90$ (nearly the same)	20.5%
pairs with $r \geq 0.999$ (identical)	3.7% (45 pairs)

Read this honestly: personalisation is real but not uniform. A quarter of persona pairs rank a video's shots very differently, and 15% are actively opposed -- that is real signal for Phase 9. But a fifth of pairs are near-identical, meaning the teacher collapses two distinct personas onto one ranking for a meaningful minority of cases. Phase 11's persona-sensitivity test should quantify whether divergence tracks persona semantic distance, and this table is the baseline it must beat.

Against the generic human gtscore (n=547): mean Spearman +0.147, sd 0.277, range [-0.67, +0.78]; 65.6% of persona labels have $|r| < 0.3$. So the persona labels are largely independent of the generic label rather than a relabelling of it -- which is the point of the method. Mildly positive on average, as expected: some shots are uninteresting to everyone.

Within-vector spread: gtscore std mean 0.266, min 0.064 -- no near-constant vectors survived.

Implementation decisions

- Domain pairing: a persona only scores videos of its own domain (Phase 6, gap G2).
 - response_format, not guided_json (silently ignored on vLLM 0.19.1).
 - Flat numeric array with minItems/maxItems -- prevents empty arrays, ~15x cheaper in tokens than array-of-objects.
 - k=3 self-consistency, averaged per chunk (runbook's first-listed quality lever).
 - Cache key = md5(dataset, video, persona, model, PROMPT_VERSION, chunk span, repeat).
- Bump PROMPT_VERSION to invalidate. Corrupt entries are deleted and recomputed.
- Persona files contain only their domain's videos -- 721 MB instead of ~11 GB, which matters given ~19 GB of home quota.

Reproduce

```
python src/extract_shot_frames.py --dataset tvsum summe      # ~6 min, cached
source /var/tmp/akn57/envs/pvs-vllm/bin/activate             # server must be up
python src/score_teacher.py --k 3 --workers 10               # ~75 min, resumable
source env/activate-train.sh
python src/build_persona_h5.py                               # ~2 min
```

Re-running costs nothing: every chunk call is cached and every completed pair is skipped.

For Phase 8/9

- Persona files are keyed data/persona_h5/<dataset>/<domain>/<persona_id>.h5; each group carries query, persona_id and domain attrs, and the file carries them too.
- Features are 512-d CLIP -> DSNet must run with --num-feature 512.
- 547 (video, persona) training vectors is a modest dataset; expect the query head to need regularisation.
- The 20.5% of near-identical persona pairs is the ceiling on how much a query-conditioned model can differentiate. Worth stating in the dissertation as a limitation of the teacher, separable from the student's ability to learn.

Phase 8 -- Query conditioning for DSNet

Date: 2026-08-12.

What changed

src/query_head.py adds QueryConditioner, fused into the frame features at the top of DSNet's forward. The persona query is encoded once by CLIP's text encoder; CLIP's shared image/text space is what makes a query comparable to a frame feature at all (the reason Phase 4 exists).

Total change to DSNet for this phase: 18 insertions, 6 deletions across two files. Cumulative DSNet diff including the Phase 2 compatibility renames: 32 insertions, 20 deletions across 5 files. Patch: results/phase8/dsnet_query_head.patch (base commit 1804176).

The architectural delta is kept minimal on purpose. The contribution of this project is the supervision; a larger architectural change would make it impossible to attribute a Phase 9 result to persona labels rather than to a better network.

Insertion

```
def forward(self, x, query_emb=None):
    qcond = getattr(self, 'qcond', None)
    if qcond is not None and query_emb is not None:
        x = qcond(x, query_emb)
    ...                # everything below is untouched DSNet
```

__init__ is not modified; the conditioner is attached externally (model.qcond = QueryConditioner(...)). So an unmodified DSNet, or one called without a query, is byte-for-byte the original code path -- which is what keeps the Phase 2/3/4 baselines valid.

Identity initialisation -- a correction to the runbook

The runbook's checkpoint says to verify the original model is recovered "when query_emb=0 and mode="concat" with zero-init proj bias". That cannot hold. For concat, $\text{proj}([f; 0]) = W_f f + b$; zeroing only b leaves $W_f f$, and W_f is randomly initialised -- a random linear map of the features, not the features.

Both modes are therefore initialised to the exact identity:

mode	init	at init
concat	$W = [I \ 0]$	$0], b = 0$
film	gamma weights 0 / bias 1, beta weights 0 / bias 0	$1*f + 0 = f$

Why this is better than what was asked for:

- the checkpoint becomes genuinely verifiable -- bit-identical, not approximately equal;
- training starts from exactly the Phase 2 baseline rather than a randomly perturbed version of it, so the optimiser never has to first undo random damage;
- if persona conditioning helps nothing, the model can remain at identity -- a clean null result instead of a confounded one.

Checkpoint results

```
=== anchor-based ===
concat identity-init: max|diff| = 0.000e+00    bit-identical: True
film   identity-init: max|diff| = 0.000e+00    bit-identical: True
gradient reaches the QUERY half of proj: 2.11e+04 (yes)
=== anchor-free ===
concat identity-init: max|diff| = 0.000e+00    bit-identical: True
film   identity-init: max|diff| = 0.000e+00    bit-identical: True
```

```
gradient reaches the QUERY half of proj: 1.47e+03 (yes)
```

The gradient check matters as much as the identity check: an identity init could have been dead (zero gradient on the query path, query permanently ignored). It is not, because the query input is non-zero, so $dL/dW_q = \delta(x) q \neq 0$.

End-to-end on real persona data (two tvsum:VT personas, same video):

```
at identity init, two personas give identical output: True (query ignored until trained)
after perturbing ONLY the query weights: max|diff| = 5.99e-03
-> the architecture CAN express persona-dependent predictions
```

Query embeddings

```
data/query_emb.npz (128, 512) float32, unit-normalised, ViT-B-32/laion2b_s34b_b79k
pairwise cosine mean 0.352, max 0.849
```

Unit-normalised to match the scale of CLIP image features. The similarity statistics match the Phase 6 audit exactly, as they should -- same encoder, same strings.

Keyed by persona_id, which is also stored as an attr on every persona h5 group, so Phase 9's loader can join them without any filename parsing.

Reproduce

```
source env/activate-train.sh
python src/query_head.py --precompute --out data/query_emb.npz
## verification
python - <<'EOF'
import sys, torch; sys.path[:0]=["third_party/DSNet/src","src"]
from modules.model_zoo import get_model
from query_head import QueryConditioner
m = get_model("anchor-based", base_model="attention", num_feature=512,
              num_hidden=128, num_head=8, anchor_scales=[4,8,16,32]).eval()
x, q = torch.randn(1,40,512), torch.randn(512)
base = m(x); m.qcond = QueryConditioner(512,512,"concat").eval()
assert all(torch.equal(a,b) for a,b in zip(base, m(x,q)))
print("identity check OK")
EOF
```

For Phase 9

- --num-feature 512 on every run (CLIP features).
- The DSNet data loader must be extended to return the query embedding per sample:

read persona_id from the h5 group attrs, look it up in data/query_emb.npz, and pass it to `model(x, query_emb) / model.predict(seq, query_emb)`.

- mode is the Axis-1 ablation: concat (default) vs film vs none.

none with persona labels isolates "supervision" from "query".

- Planned replacement for the dropped CLIP-It control (Phase 3): the same

query-conditioned model trained on the stock generic gtscore. Same architecture, query head on, generic supervision -- isolates "reads a query" from "persona supervision" within one model rather than across two codebases.

Phase 9 -- Training the student on synthetic labels

Superseded headline: the concat results below were produced by an architecture that cannot personalise. See FINAL RESULT (FiLM) at the bottom, which is the configuration to report.

Date: 2026-08-13. TVSum, anchor-free DSNet + query head, CLIP-512 features. 5 splits (DSNet's own), 313 train / 80 test (video, persona) samples per split.

Headline result

Spearman between the model's per-frame scores and the held-out persona labels (every condition measured on the same yardstick):

Condition	labels	query	mean rho	sd
A (the method)	persona	yes	+0.2154	0.056
B	persona	no	+0.1995	0.061
C	generic	yes	+0.2186	0.049

Paired per-split comparisons (same splits, same data):

comparison	mean delta	splits positive	paired t
A - B (what the query adds)	+0.0159	5/5	p = 0.037
A - C (what persona supervision adds)	-0.0033	3/5	p = 0.866

Persona supervision gives no measurable benefit over generic supervision. The query contributes a small but consistent gain -- which the diagnostic below shows is not actually personalisation.

Reference points (split 0):

random scores	+0.004	
untrained model	-0.087	
trained model	+0.192	
generic human gtscore	+0.213	<- a trivial predictor still beats the model

The decisive diagnostic: the student does not personalise

Same video, different persona queries, correlation between the model's outputs:

checkpoint	mean l	W_queryl	spearman between personas	min
main run A (lr mult 1)	7.3e-04 (~0)	+0.9999	1.000	
lr mult 10	3.3e-02	+0.9682	0.850	
lr mult 100	5.1e-02	+0.9984	0.987	
lr mult 1000	1.0e-01	+0.9993	0.997	

(1.0 means the query changes nothing.)

Two different personas receive essentially the same summary. So A ~ C because both are effectively plain DSNet, and the +0.016 "query benefit" is a negligible perturbation, not personalisation.

Cause, in two parts:

1. The query head never trained. Identity initialisation (Phase 8) starts the query

weights at exactly zero -- correct for the checkpoint, but it means the head must travel further than any other parameter. Model selection picks epoch 0-2 because the detection task overfits almost immediately on 313 samples, so early stopping fires before the query path leaves zero ($|W_{\text{query}}| = 7e-04$).

2. Forcing it to train does not fix the ordering. With a larger lr on the conditioner the weights do grow (up to $1e-01$), but sensitivity does not increase monotonically: 10x gives the most differentiation, 100x and 1000x give less. The weights grow into a direction that shifts all frames roughly equally, and Spearman is rank-based, so a uniform shift changes nothing. The model is not learning persona-specific ordering.

Why this is a real finding, not just a bug

The teacher does personalise (Phase 7): pairwise Spearman between two personas' labels for the same video has median 0.64, with 27.3% of pairs below 0.3 and 15.3% negative. The student, trained on exactly those labels, collapses to 0.97-1.00.

The signal exists in the labels and is lost in the student. Most plausible reasons, in order:

1. Data scale. 313 training samples = 40 videos x ~8 personas. Learning a query -> score mapping that generalises to unseen videos is far harder than fitting per-video importance, and the latter reduces the loss faster.

2. The persona-specific component is small. Phase 7: persona labels correlate +0.147 with the generic human label, and 20.5% of persona pairs are already near-identical in the teacher's own output. The learnable persona-specific residual is thin.

3. Objective mismatch. DSNet's detection losses dominate; the frame-level ranking objective (the one matching the metric) had to be weighted 10x before it mattered at all, and even then the model prefers the generic solution.

Tuning that did work

change	rho
lambda_score = lambda_rank = 1 (runbook default)	+0.1401
lambda_score = lambda_rank = 10	+0.1919
rank-only, lambda 20	+0.1842

DSNet's detection losses run ~2.5 while MSE was ~0.11 and rank ~0.66, so the objectives matching the primary metric were being drowned out. Weighting them 10x gave +37% relative.

Learning rate: $5e-5$ (+0.192, peak ep0), $2e-5$ (+0.186, ep2), $1e-5$ (+0.184, ep3), $5e-6$ (+0.180, ep12). Lower lr moves the optimum later but reaches the same ~0.19 ceiling -- an optimisation-independent limit.

Two methodological traps avoided (and one caught late)

Video-level splits, not pair-level. Each video appears with ~8 personas; a random split over pairs would place the same video in train and test, letting the model memorise per-video importance and score well while ignoring the query entirely. DSNet's own 5 splits are reused so results stay comparable with Phases 2-4.

Persona-aware model selection. DSNet keeps the epoch with max F1 against user_summary -- the generic human annotation, identical across a video's personas. Selecting on that while training on persona labels would pick the epoch that best ignores the persona. Selection uses Spearman against held-out persona labels instead.

Caught late: the control was measuring itself. evaluate() originally scored each model against s.gtscore, i.e. its own training target, so condition C (generic labels) reported rho +0.4650 and appeared to beat the method 2.4x. Fixed by splitting the roles: gtscore is the training target (varies by condition), eval_gtscore is the measurement target and is always the persona label. Uncorrected, Phase 9 would have concluded that generic supervision beats persona supervision -- an inverted result.

Implementation notes

DSNet does not regress gtscore: it converts it into a binary 15%-budget keyshot target and trains focal / IoU / centreness losses. Persona labels therefore enter through which shots become positives, so the pipeline consumes them correctly

unmodified. The runbook's "MSE against persona gtscore" was implemented as an auxiliary loss on `pred_cls` (the anchor-free per-frame score, which is exactly what Phase 10 measures).

Best config: `--loss mse+rank --lambda-score 10 --lambda-rank 10 --lr 5e-5 --qcond concat`.

What to do next

The honest framing for the dissertation: the pipeline works end-to-end and the teacher personalises; the student at this data scale does not learn to. That is a legitimate result, and it is diagnosed rather than merely observed.

Options, roughly by expected value:

1. Evaluate on QFVS (Phase 10.3). The runbook is explicit that per-query Oracle summaries are "the test TVSum/SumMe can't give you". TVSum has no per-persona ground truth, so rho-against-teacher-labels is a proxy for a proxy.
2. Run the Phase 11.2 persona-sensitivity test properly, and report the teacher's divergence (median 0.64) beside the student's (0.97-1.00). The gap between them is itself the finding.
3. Increase persona-specific signal: more personas per video, more videos, or a stronger teacher (a 4-bit AWQ 32B, since FP8 is impossible on sm_86).
4. Try FiLM conditioning (Axis 1) -- multiplicative modulation may produce rank-changing effects where additive concat produced a near-uniform shift.
5. Consider conditioning earlier or at multiple depths rather than once at the input.

Files

```
src/train_persona.py          trainer (3 label modes, 3 conditioning modes)
models/p9_persona_concat/    condition A checkpoints + results.json
models/p9_persona_noquery/   condition B
models/p9_generic_concat/    condition C
results/phase9/main_AB.log,  loss_sweep.log, lr_sweep.log,
                             qcond_lr_sweep.log
```

FINAL RESULT (FiLM) -- the configuration to report

```
--qcond film --qcond-lr-mult 100 --loss mse+rank
--lambda-score 10 --lambda-rank 10 --lr 5e-5 --max-epoch 20
```

Why concat had to fail

concat computes $out = W_f f + W_q q + b$. The query q is constant across all frames of a video, so $W_q q$ is one fixed vector added identically to every frame. It translates the sequence in feature space but leaves relative differences between frames almost unchanged -- and rank correlation only sees relative order.

concat was therefore structurally incapable of expressing a persona-specific reordering. Raising its query-head lr to 1000x grew the weights to 1e-01 and still left inter-persona correlation at 0.999.

FiLM computes $\gamma(q) * f + \beta(q)$. The multiplicative term scales each dimension differently, so frames with different feature values move by different amounts -- which genuinely reorders them.

Axis 1 (concat vs FiLM) is therefore not a hyperparameter; it is the difference between a model that cannot personalise and one that can. The runbook lists concat as the default; on this evidence FiLM should be.

Results, 5 splits, TVSum

Accuracy (Spearman vs held-out persona labels):

condition	labels	mean rho	sd
A2 (the method)	persona	+0.2237	0.044
C2 (control)	generic	+0.2199	0.057

paired A2 - C2 = +0.0037, 3/5 positive, $p = 0.872$ -> no accuracy difference.

Persona sensitivity (correlation between the model's outputs for two DIFFERENT personas on the same video; 1.0 means the query is ignored):

model	mean	median	min	% pairs < 0.9
A2 persona + FiLM	+0.9080	+0.9397	-0.289	27.4%
C2 generic + FiLM	+0.9829	+0.9897	+0.614	1.0%
teacher (Phase 7)	-	+0.64	-	27.3% (below 0.3)

The finding

Persona supervision produces a persona-sensitive model -- 27x more differentiated outputs than the control -- while producing no gain in aggregate accuracy.

Both halves matter:

- The control behaves exactly as theory predicts. C2 was trained with queries on labels that do not depend on them, and correctly learned to ignore the query (1.0% differentiated). That validates the sensitivity measurement itself.

- A2 learned persona-conditioned behaviour from persona labels alone, and its share of differentiated pairs (27.4%) closely matches the teacher's own (27.3%). The student inherited roughly the proportion of persona-differentiated cases its supervision contained, expressed more weakly (median 0.94 vs the teacher's 0.64).

Why rho cannot see this: persona labels correlate +0.147 with generic importance (Phase 7), so most of what rho measures is the generic component. Predicting generic importance captures most of the metric; the persona-specific residual is small in rho but is exactly what A2 captures behaviourally.

Implication for evaluation: agreement metrics (F1, rank correlation against a single label) structurally cannot detect personalisation. The counterfactual divergence test -- runbook Phase 11.2 -- is required. This result is the empirical justification for that.

Phase 9 checkpoint

"a trained personalised model that, given the same video and two different persona queries, produces two visibly different summaries"

PASSES with FiLM: 27.4% of persona pairs differ substantially, minimum correlation -0.289 (two personas receiving near-opposite selections of the same video). It FAILS with concat (0.0% of pairs, correlation +0.9999).

Checkpoints: models/p9_persona_film/, models/p9_generic_film/. Still TVSum only -- SumMe was not trained in Phase 9.

Phase 10 -- Evaluation

Date: 2026-08-13. Primary metric: rank correlation (Otani et al. 2019 protocol).

Why rank correlation is primary

Phase 3 measured a random-score model at 56.42 F1 on TVSum against published DSNet's 62.09. F1 is computed after KTS segmentation and a knapsack filling a 15% budget, so it largely measures the pipeline, not the model. Rank correlation compares the predicted ordering against human orderings and has no such shortcut.

Protocol, implemented per dataset (src/eval_rank.py):

- TVSum -- correlate against each of the 20 annotators, then average. Uses ydata-tvsum50-anno.tsv (per-frame 1-5 Likert). Verified: that file's length equals n_frames, and the h5 gtscore is exactly its mean sampled at picks (Spearman 1.0000).
- SumMe -- average the 15-18 binary annotator selections first, then correlate once.

Reversing these is the runbook's listed failure mode ("rank correlation ~ 0 -- protocol mismatch"). The random floor coming out at $\tau = +0.0009$ (TVSum) and -0.0029 (SumMe) confirms the implementation is sound.

10.1 Main results -- TVSum

method	τ	ρ	F1
Human ceiling (leave-one-out)	0.3139	0.3957	-
Random floor	0.0009	0.0013	56.42
Generic human gtscore (reference)	0.3782	0.4732	-
DSNet AF (GoogLeNet-1024)	0.0898	0.1177	61.16
DSNet AF (CLIP-512)	0.0826	0.1079	61.29
Ours: persona + FiLM	0.1581	0.2059	-
Control: generic + FiLM	0.2701	0.3488	-

Two findings

1. The auxiliary ranking objective is worth ~3x on the primary metric. Standard DSNet training reaches τ 0.083-0.090. The same architecture trained with the Phase 9 frame-level mse+rank objective reaches 0.158 (persona labels) and 0.270 (generic labels). DSNet's detection losses optimise a binary 15%-budget target, which is only loosely related to the ordering that rank correlation measures; adding an explicit ranking term on pred_cls closes that gap. This is runbook Axis 2's rationale confirmed empirically, and it is a result in its own right.

2. The metric structurally penalises personalisation. The control (generic labels) scores 0.270 versus our 0.158. That is expected and is not evidence the method is worse: the reference is the generic human annotation, so a model that deliberately deviates per persona is scored down for exactly the behaviour it was built to have. The control optimises precisely what this metric rewards.

TVSum has no per-persona ground truth, so τ -against-generic-annotators cannot separate "personalised" from "wrong". This is the concrete reason the runbook schedules QFVS (10.3) and the persona-sensitivity test (11.2) -- see below.

Note the human ceiling (0.3139) sits below the generic gtscore reference (0.3782) because the latter is the mean of all 20 annotators, while the ceiling correlates each annotator against the mean of the other 19. The ceiling is the honest bar for a model.

10.1 Main results -- SumMe

method	tau	rho	F1	note
Human ceiling	0.2966	0.3299	-	
Random floor	-0.0029	-0.0039	41.49	
Generic human gtscore	1.0000	1.0000	-	circular -- SumMe's gtscore is derived from user_summary
DSNet AF (GoogLeNet-1024)	0.1002	0.1318	51.31	
DSNet AF (CLIP-512)	0.1070	0.1415	50.64	
Ours: persona + FiLM	0.0256	0.0329	-	TVSum-trained, zero-shot transfer
Control: generic + FiLM	0.0920	0.1236	-	TVSum-trained, zero-shot transfer

Two rows must not be read naively:

- the "generic human gtscore" row is self-comparison (SumMe's h5 gtscore is built from user_summary), so 1.0 is an artefact, not evidence;
- our models were never trained on SumMe (Phase 9 was TVSum-only), so those rows measure cross-dataset transfer. The drop from 0.158 to 0.026 says the model does not transfer, which is unsurprising given 16 domains and 313 training samples.

Gap to close: Phase 9 should be re-run on SumMe for a like-for-like row.

10.2 F1 (secondary)

Reported in the table above and in results/baselines.csv, always beside the random floor (TVSum 56.42, SumMe 41.49). Never lead with it: the floor is within ~5 points of every published model, so F1 has very little discriminative range on these datasets.

10.3 QFVS -- BLOCKED, not run

data/qfvs/ is empty; the UT Egocentric videos and per-query Oracle annotations were never downloaded. This is the only protocol in the runbook with genuine per-query ground truth, and Phase 10.1 above shows precisely why it matters: on TVSum/SumMe a personalised model can only be penalised for personalising.

This is the single highest-value remaining experiment. Cost: 4 UTE videos (10-17 h each) plus the QFVS annotation set, and a separate evaluation harness.

10.4 Synthetic-label validation (the crux)

Does the teacher's persona score agree with real humans at all? If not, the persona h5 files are elaborately generated noise.

	tau vs human annotators
Random floor	+0.0009
Teacher persona labels	+0.0802 +/- 0.1116
Human ceiling	+0.3139
Generic human gtscore	+0.3782

n = 393 (video, persona) pairs; 79.9% positive, range [-0.296, +0.403].

The labels are not noise: 80x the random floor, four fifths positive. But they capture only 26% of human-ceiling agreement. Part of that shortfall is intentional -- persona labels are supposed to deviate from generic judgment -- and part is presumably teacher error. Generic human annotations cannot separate the two. QFVS can.

Checkpoint

results/main_table.csv exists with tau/rho (primary), F1 (secondary), and random-floor and human-ceiling rows for both

datasets. Met, with the caveats above: PGL-SUM and CLIP-It rows are absent (CLIP-It was dropped in Phase 3; PGL-SUM's per-frame scores could be added from its saved epoch JSONs).

Honest summary

- The pipeline runs end to end and every metric is bracketed by a floor and a ceiling.
- The frame-level ranking objective is a real, transferable gain ($\sim 3\times$ tau over stock

DSNet training).

- The teacher's synthetic labels carry genuine human-aligned signal (26% of ceiling).
- Whether the persona-specific component is right cannot be tested on TVSum/SumMe,

because their ground truth is generic. On this evidence the personalised model looks worse than a generic one by construction of the metric.

- Phase 11.2 (persona sensitivity) and Phase 10.3 (QFVS) are the two evaluations that can actually adjudicate the central claim.

Files

src/eval_rank.py	Otani-protocol evaluator (both datasets)
results/main_table.csv	the Phase 10 checkpoint table
results/phase10/rank_tvsum.json	per-method tau/rho
results/phase10/rank_summe.json	

Phase 11 -- Ablations and the persona-sensitivity test

Date: 2026-08-13. TVSum, 5 splits, persona labels, lr 5e-5, lambda_score = lambda_rank = 10, 20 epochs. Each axis varies exactly one thing from that default.

Two metrics per row:

- rho -- Spearman against held-out persona labels (accuracy).
- persona sensitivity -- mean Spearman between the model's outputs for two DIFFERENT personas on the same video. 1.0 means the query is ignored; lower means the model actually personalises. Accuracy alone cannot tell these apart.

11.1 Axis 1 -- query mechanism (the decisive axis)

variant	rho	sd	persona sens.	% pairs < 0.9
FiLM (default)	+0.2237	0.044	+0.9080	27.4%
concat	+0.2036	0.059	+0.9923	0.4%
none (no query at all)	+0.2025	0.061	n/a	n/a

concat is functionally equivalent to having no query. It scores +0.2036 against no-query's +0.2025 -- a difference of 0.001 -- and changes its output for only 0.4% of persona pairs.

This is structural, not a tuning failure. concat computes $W_f f + W_q q + b$; the query q is constant across all frames of a video, so $W_q q$ is a single fixed vector added identically to every frame. It translates the sequence in feature space and leaves the relative ordering essentially untouched -- and rank metrics only see relative order. FiLM's $\gamma(q) * f$ scales each dimension, so frames with different feature values move by different amounts, which genuinely reorders them.

The runbook specifies concat as the default (Axis 1, "start with concatenation"). On this evidence that default cannot personalise and FiLM should be the default.

11.1 Axis 2 -- loss (all with FiLM)

variant	rho	sd	persona sens.	% pairs < 0.9
mse+rank (default)	+0.2237	0.044	+0.9080	27.4%
rank only	+0.2209	0.052	+0.9351	16.6%
mse only	+0.2117	0.039	+0.9151	32.2%
none (DSNet losses only)	+0.1648	0.036	+0.9499	10.6%

The auxiliary frame-level objective is worth +0.059 rho (+36% relative) over DSNet's detection losses alone. DSNet's losses optimise a binary 15%-budget target, which is only loosely coupled to the ordering the primary metric measures.

Note the accuracy/sensitivity trade-off: mse+rank is the most accurate, but mse alone is the most persona-sensitive (32.2% of pairs). If personalisation is the goal rather than agreement with teacher labels, mse is arguably the better choice.

11.1 Axis 3 -- teacher self-consistency (k=1 vs k=3)

Measured directly on label quality (Otani tau against the 20 human annotators), which isolates the teacher without needing a training run. k=1 is recoverable for free from the cached k=3 calls (repeat 0).

tau vs human annotators

k=1	+0.0823 +- 0.109
k=3	+0.0802 +- 0.111
paired k=3 - k=1	-0.0021, 50% positive, p = 0.382

k=1 and k=3 labels agree at tau +0.799 -- averaging changes the labels but not their human alignment.

Self-consistency produced no measurable benefit at 3x the cost (6,576 teacher calls versus 2,192). The runbook lists it as the quality lever to apply first; on this evidence it should be dropped, and the ~4,400 saved calls spent on more personas or more videos.

11.1 -- axes NOT run, with reasons

axis	why not
teacher 8B vs 32B	FP8 is impossible on sm_86 (Phase 1); would need a 4-bit AWQ build
modality visual+ASR	no transcript pipeline exists in this project
cross-attention conditioning	not implemented
KL loss	not implemented
frames vs captions teacher input	not implemented

11.2 Persona sensitivity -- the counterfactual test

Phase 10 showed TVSum/SumMe ground truth is generic, so agreement metrics penalise a model for personalising. This test needs no per-persona ground truth: fix the video, change only the persona, measure whether the summary changes -- using Jaccard over the actually selected shot sets (the real KTS -> knapsack pipeline).

Crucially, changing is not enough. The claim requires that divergence track persona meaning: similar personas -> similar summaries.

persona distance = $1 - \cos(\text{query_emb}_i, \text{query_emb}_j)$ (CLIP text space) summary divergence = $1 - \text{Jaccard}(\text{selected shots})$

condition	n	mean divergence	% pairs differ	corr(distance, divergence)	p
TEACHER labels (upper bound)	1372	0.4811	92.3%	+0.0222	0.410
Ours: persona+FiLM	1372	0.1451	57.0%	+0.0898	0.001
Control: generic+FiLM	1372	0.1082	45.3%	+0.1564	0.000
Ours + shuffled queries	1372	0.1787	71.4%	+0.0333	0.218

The central finding

The teacher's labels diverge enormously but not semantically. Divergence is 0.48 and 92% of persona pairs differ -- yet the correlation with persona semantic distance is +0.022, not significant (p = 0.41). Qwen gives different personas different scores, but semantically similar personas do not receive more similar scores.

The supervision is differentiated but not semantically organised. A student cannot learn a distance -> divergence relationship that its labels do not contain. This is the precise, upstream explanation for the weak Phase 9 and Phase 10 personalisation results, and it locates the limitation in the teacher, not the student or the architecture.

Plausible cause: the Phase 7 scoring prompt rates shots for one persona in isolation. Nothing in it encourages the model to place similar personas near each other in score space. A prompt that scores several personas jointly, or an explicit consistency constraint across personas, is the obvious next experiment.

The positive signal

The shuffled-query condition (not in the runbook, added because a reviewer will ask whether the model responds to persona meaning or merely to some vector changing):

- correct personas: $r = +0.0898$, $p = 0.001$
- shuffled personas: $r = +0.0333$, $p = 0.218$

The distance-divergence relationship exists only when the model receives the matching persona. The model does respond to persona meaning -- weakly, but measurably, and the effect is not an artefact of perturbation.

Two caveats stated rather than hidden

1. The control correlates higher (+0.156) than the method (+0.090). This is probably not in its favour: its divergence magnitude is lower (0.108 vs 0.145), and a model trained on query-independent labels plausibly learned a smooth function of the query embedding. Smooth responses correlate well with distance while carrying no persona information, so this metric partly rewards smoothness rather than correctness.
2. The control diverges at all (45.3% of pairs) despite its score-level correlation being 0.983. Jaccard over selected shots amplifies small score differences, because knapsack selection is discrete: a tiny score change can flip which shot is chosen. Score-level and summary-level sensitivity are different measurements and should be reported separately.

11.3 User study -- not run

Requires 10-15 human participants; outside what can be executed here. The generated material (persona description + resulting summary) is available from the persona h5 files and trained checkpoints.

Summary for the write-up

1. Axis 1 is not a hyperparameter. concat cannot personalise (0.4% of pairs); FiLM can (27.4%). The runbook's default is the wrong one.
2. The auxiliary ranking objective is worth +36% rho over stock DSNet training.
3. Self-consistency ($k=3$) is not worth 3x the teacher cost.
4. The teacher's persona scores are differentiated but not semantically organised ($p = 0.41$). This is the key limitation of the current method and the clearest direction for future work.
5. The student does respond to persona meaning ($p = 0.001$, and it vanishes under shuffled queries) -- weakly, bounded by (4).

Files

```
src/persona_sensitivity.py      the counterfactual test
results/phase11/sensitivity.json 11.2 results
results/phase11/axis1.log, axis2.log ablation training logs
models/ab_qcond_concat, ab_qcond_none, ab_loss_{none,mse,rank}
```

Phase 13 -- Raw video -> personalised summary (not in the runbook)

Date: 2026-08-13. src/infer_personalised.py.

The runbook stops at evaluation; it never produces a usable system. This phase closes that gap: an arbitrary .mp4 plus a free-text preference in, a summary .mp4 out.

```
python src/infer_personalised.py \
  --source video.mp4 \
  --query "close-ups of hands working with tools, and the finished result" \
  --save summary.mp4

## demonstrate personalisation: two preferences, same video, reports the overlap
python src/infer_personalised.py --source video.mp4 \
  --query "the athlete mid-jump and landing" \
  --query2 "people being interviewed and talking to camera" \
  --save demo.mp4
```

The query is not restricted to the 128 generated personas. CLIP maps arbitrary text into the same space, so any sentence works; the model never sees a persona id at inference.

What it does

```
raw mp4 -> sample every 15th frame -> CLIP-512 features
        -> KTS shot boundaries (no annotations needed)
        -> FiLM query conditioning + DSNet anchor-free
        -> knapsack at 15% budget -> summary mp4
```

DSNet's own infer.py already had this shape. Three changes were required: CLIP-512 features instead of GoogLeNet-1024, threading query_emb into model.predict, and encoding the user's text with CLIP's text encoder. PyAV is used for both decode and encode, so OpenCV is not a dependency.

Working demo

Source: cjbttmSLxQ4.mp4 (TVSum video_21, parkour) -- in split 0's test set, unseen by the model. 647 s, 19,406 frames.

query	shots	frames	duration
"the athlete mid-jump and landing, the body in motion during a vault"	45/175	2,910	97.0 s
"people being interviewed and talking to camera, the city background"	44/175	2,910	97.0 s

Jaccard overlap 0.679 -- 9 shots unique to A, 8 unique to B, 36 shared. Both summaries are exactly 15.0% of the source, as intended.

Three real bugs found and fixed

1. KTS on un-normalised CLIP features degenerates. DSNet L2-normalises its GoogLeNet descriptors before building the KTS kernel; our CLIP features are deliberately un-normalised (to match training), which makes kernel values ~120x larger and swamps cpd_auto's complexity penalty. Result: 635 "shots" for 636 sampled frames -- one shot per frame. Fixed by normalising for the kernel only, leaving the model input untouched.
2. KTS vmax needed retuning for CLIP. With vmax=1.0 (DSNet's value) the same video gave 22 shots against the dataset's 64, and the coarse selection was identical for both queries. Measured sweep:

vmax **shots found (dataset truth: 64)**

1.00	22
0.50	67
≤ 0.20	319 (maximum)

vmax=0.5 is now the default and reproduces the dataset's ~1 shot per 10 sampled frames.

3. Non-contiguous frames inherit their source timestamps. Kept frames are scattered through the source, so writing them with original PTS produced a summary that reported 376 s for 1,425 frames (~59 s of content) and would play with long freezes. Clearing pts instead makes the muxer reject non-monotonic DTS. Fixed by assigning pts from the output frame index with an explicit time_base.

How reliably does it personalise?

Two contrasting, domain-appropriate queries per video, on five unseen split-0 test videos, using the dataset's own shot boundaries:

video	domain	shots	A-only	B-only	Jaccard
video_16	making sandwich	64	3	4	0.682
video_43	bike tricks	33	2	2	0.714
video_21	parkour	130	10	10	0.500
video_35	flash mob	30	1	1	0.818
video_38	beekeeping	20	2	2	0.500

5/5 produced different summaries, mean Jaccard 0.643.

Caveats to state in a demo

- Shot boundaries matter more than expected. The persona signal is strong enough to reorder scores but only marginally strong enough to flip a discrete selection. On video_16, the dataset's 64 shots gave Jaccard 0.682 while our KTS's 67 shots gave 1.000 (identical) -- same count, different boundary placement. This is a direct consequence of the weak persona signal measured in Phase 11, and it means demo quality varies by video.

- Generalisation is limited. Trained on 40 TVSum videos across 16 domains. Expect sensible behaviour on similar content (how-to, sports, events) and unpredictable behaviour elsewhere. Phase 10 measured zero-shot SumMe transfer at tau 0.026 vs 0.158 in-domain.

- A demo is not evidence. The results chapter rests on Phases 10-11. A compelling side-by-side is persuasive in a viva, but is a single sample.

- The checkpoint used is models/p9_persona_film/split0.pt, trained on split 0's 40 training videos. For a deployed demo, train one model on all 50.

Files

src/infer_personalised.py	the pipeline
results/phase13/demo_parkour_A.mp4	"athlete mid-jump and landing"
results/phase13/demo_parkour_B.mp4	"people being interviewed"